SYSTEM DESIGN FOR DATA ENGINEERING

THE 6 STEP FRAME WORK

1. **Requirements Gathering** : Focus on understanding user needs, latency SLAs, and data volumes before drawing any architecture diagrams.
2. **Pipeline Design** : Determine whether to use batch, streaming, or a hybrid (Lambda/Kappa) approach based on the requirements.
3. **Data Modeling** : Use the *Medallion Architecture* (Bronze, Silver, Gold layers) and choose appropriate schemas like *Star Schema* or *One Big Table (OBT)*.
4. **Storage & File Formats** : Select optimal file formats like *Parquet* or *Avro* and implement storage optimization strategies like partitioning and *Delta Lake*.
5. **Data Quality & Observability** : Ensure reliability through data quality dimensions, *Data Contracts*, and pipeline-level monitoring.
6. **Scalability, Backfills & DataOps** : Design for idempotency and resilient backfill strategies to handle failures gracefully.

Requirements Gathering — as explained in the video

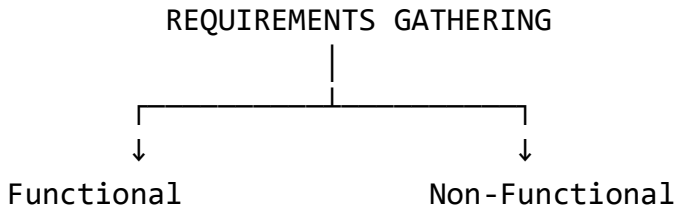
The basic idea is:

Before designing anything, understand exactly what you are being asked to build.

You should **not immediately say Kafka, Spark, Airflow, Snowflake, Databricks, etc.**

The video places requirements gathering as **Step 1 of the 6-step Data Engineering System Design framework**.

The structure is:



```
Requirements            Requirements
      |                       |
      ↓                       ↓
 WHAT to build          HOW WELL it works
```

1. Functional Requirements

Question you are answering:

"What does the system need to do?"

These describe the **actual functionality** of the data system.

You need to understand things such as:

WHO – END USER LIKE MARKET ANALYST , ML TEAM ETC

WHAT -- AGGREGATION METRICS , HISTORICAL SNAP SHOT DATA

HOW -- RETRIVED THROUGH SQL? BI TOOL? REST API?

A. Who are the users/consumers?

Ask:

Who is going to consume this data?

For example:

```
Business Analysts -- batch
Data Scientists --- stream
ML Models       ---stream
BI Dashboards   ---”
Applications     ---”
```

This matters because different consumers have different requirements.

Example 1:

A BI team may need aggregated sales data every morning.

Example 2:

A fraud-detection application may need transaction data within seconds.

Those two requirements lead to completely different pipeline designs.

2. What questions must the data answer?

This is an important way of thinking about requirements in **Data Engineering System Design**.

Don't just ask:

"What data do you want?"

Ask:

"What are you going to do with the data?"

For example, suppose the interviewer says:

Design an e-commerce data platform.

You could clarify:

```
What metrics are required?

Revenue?
Orders?
Customer retention?
Product performance?
Inventory?
Real-time sales?
```

Because these determine what data needs to be collected and how it needs to be modeled.

3. Data Freshness / Latency

You need to determine:

How quickly does the data need to become available?

This is one of the requirements that heavily influences architecture.

For example:

Example 1 — Daily reporting

```
Source
  ↓
Batch ingestion
  ↓
ETL
  ↓
Warehouse
  ↓
Dashboard

Freshness: once/day
```

Example 2 — Real-time fraud detection

```
Transaction
      ↓
Streaming
      ↓
Processing
      ↓
Fraud detection
      ↓
Alert

Freshness: seconds
```

Same business domain, completely different architecture.

This is why you **cannot choose batch vs streaming before gathering requirements**.

4. Non-Functional Requirements

Now the question changes.

Functional requirements ask:

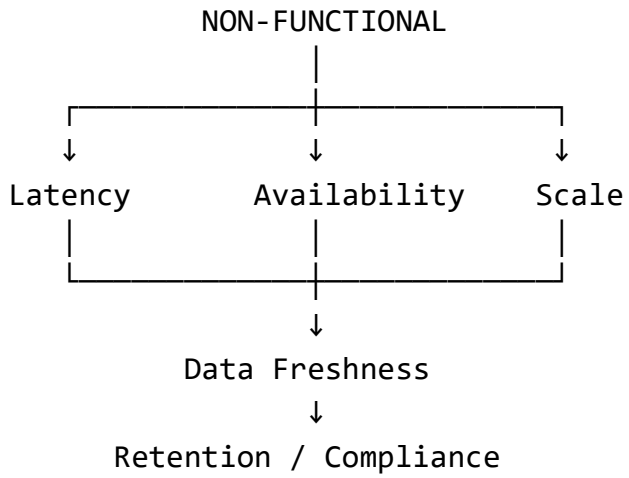
WHAT should the system do?

Non-functional requirements ask:

HOW should the system perform?

The video explicitly identifies requirements gathering as covering **functional + non-functional requirements**.

The major things you should clarify are:



5. Latency

Ask:

How quickly does the user/application need the data?

For example:

```
5 minutes
1 minute
10 seconds
1 second
100 ms
```

Do not confuse this with **data freshness**.

A useful distinction:

Freshness

How old can the data be?

Latency

How quickly does the system respond/process something?

For example:

```
Data can be 5 minutes old
but query response must be <200 ms
```

Those are two different requirements.

6. Availability

Ask:

How available does the system need to be?

For example:

```
99%
99.9%
99.99%
```

Why does this matter?

Because higher availability means you need to think about:

```
Redundancy
Failover
Replication
Disaster recovery
Fault tolerance
```

You don't need to over-engineer a system requiring a daily report that can tolerate a 2-hour outage.

7. Scale / Volume

You need to understand:

How much data are we dealing with?

Ask:

```
How many records/day?
How many events/second?
How many users?
How many sources?
How much storage?
What is the expected growth?
```

For example:

```
1 million events/day
```

versus

```
1 billion events/day
```

are not the same engineering problem.

And this is exactly why the video has a separate **Back-of-the-Envelope Calculations** section immediately after requirements gathering.

8. Data Retention

Another requirement you should establish is:

How long does the business need the data?

For example:

```
Raw data → 7 years
Aggregated data → 3 years
Operational data → 90 days
```

Retention affects:

- Storage cost
- Partitioning
- Archival
- Data lifecycle policies
- Compliance

9. Security / Compliance

Ask whether there are restrictions around the data.

For example:

```
PII?
Financial data?
Healthcare data?
GDPR?
HIPAA?
SOX?
```

Then determine:

```
Who can access it?
What data can they access?
Should sensitive columns be masked?
How long must audit logs be retained?
```

This can significantly affect the architecture.

10. The final requirements-gathering checklist

For a **Data Engineering System Design interview**, structure your questions like this:

```
STEP 1 — REQUIREMENTS GATHERING

A. Users / Consumers
   └── Who consumes the data?

B. Functional Requirements
   ├── What should the system do?
   ├── What data/use cases are required?
   └── What questions should the system answer?

C. Data Freshness
   └── How quickly must data be available?

D. Latency
   └── How quickly must queries/processes respond?

E. Scale
   ├── Records/events per day
   ├── Events per second
   ├── Number of users
   └── Expected growth
```

```
F. Availability
   └── What uptime/SLA is required? -- service level agreement

G. Retention
   └── How long must data be stored?

H. Security / Compliance
   ├── PII?
   ├── Access control?
   └── Regulatory requirements?
```

Then you **summarize the requirements back to the interviewer** before moving on.

11. Why this step is so important

The biggest lesson from the video is that **requirements drive the architecture**.

For example:

```
Requirement:
Real-time data
       ↓
Streaming ingestion
       ↓
Kafka
       ↓
Stream processing
```

versus:

```
Requirement:
Daily analytics
       ↓
Batch ingestion
       ↓
Scheduled processing
       ↓
Data warehouse
```

So if you say:

"I'll use Kafka."

before asking whether the system actually requires real-time processing, you're **solutioning before understanding the problem**.

That is exactly the mistake this framework is designed to prevent. The video then moves from requirements gathering to **back-of-the-envelope calculations**, and only after that to **pipeline design**.

The complete flow from this video

```
1. Requirements Gathering
          ↓
2. Back-of-the-Envelope Calculations
          ↓
3. Pipeline Design
   ├── Batch vs Streaming
   ├── Lambda
   └── Kappa
          ↓
4. Data Modeling
   ├── Medallion
   ├── Star / OBT
   ├── SCD
   └── Partitioning
          ↓
5. Storage & File Formats
   ├── Parquet
   ├── Avro
   └── Delta/Iceberg/Hudi
          ↓
6. Data Quality & Observability
          ↓
7. Scalability / Backfills / DataOps
```

The linked video's published chapter structure confirms these topics and their ordering.

what is idempotent pipeline

An **idempotent pipeline** is a data pipeline where **running the same input multiple times produces the same final result as running it once**.

In Data Engineering, this is critical because pipelines can fail halfway, be retried, or accidentally be triggered twice.

Simple definition

Idempotency = retrying the same pipeline/job does not create incorrect duplicate results.

Formally, if processing input X produces state Y:

```
process(X) → Y
process(X) again → Y
process(X) again → Y
```

The final state remains **Y**, rather than:

```
Y
Y + duplicate data
Y + duplicate data + duplicate data
```

Example 1: Daily sales pipeline

Suppose today's source file contains:

```
orders_2026_08_25.csv

Order_ID    Amount
101         500
102         700
103         300
```

Your pipeline loads it into a warehouse.

First execution:

```
CSV
 ↓
Transform
 ↓
Warehouse
```

```
101 → 500
102 → 700
103 → 300
```

Now suppose the pipeline crashes after loading the data, and Airflow retries the task.

Non-idempotent pipeline

The retry simply does:

```
INSERT INTO orders
SELECT * FROM today's_file;
```

Now:

```
101 → 500
102 → 700
103 → 300
101 → 500   ← duplicate
102 → 700   ← duplicate
103 → 300   ← duplicate
```

That's a problem.

Idempotent pipeline

Instead, you could use a deterministic key such as `order_id` and perform an upsert/merge:

```
MERGE INTO orders target
USING today's_file source
ON target.order_id = source.order_id
WHEN MATCHED THEN
    UPDATE SET amount = source.amount
WHEN NOT MATCHED THEN
```

```
    INSERT (order_id, amount)
    VALUES (source.order_id, source.amount);
```

Running it once:

```
101, 102, 103
```

Running it again:

```
101, 102, 103
```

No duplicates.

Example 2: Daily partition replacement

Suppose your pipeline processes:

```
2026-08-25
```

and writes:

```
/data/sales/date=2026-08-25/
```

Instead of blindly appending every retry, the pipeline can:

```
1. Delete/replace the existing 2026-08-25 partition
2. Reprocess the source data
3. Write the complete partition
```

First run:

```
date=2026-08-25
→ 10 GB
```

Pipeline fails.

Retry:

```
replace date=2026-08-25
→ write 10 GB
```

Final result:

```
date=2026-08-25
→ 10 GB
```

Not:

```
20 GB
```

This is a common pattern for **batch data pipelines**.

Why do we need idempotency?

Because distributed pipelines **will fail and retry**.

For example:

```
Source
  ↓
Extract
  ↓
Transform
  ↓
Load
  ↓
Database
```

Suppose the database load succeeds:

```
Load → SUCCESS
```

but the worker crashes before acknowledging the success:

```
Database successfully updated
             ↓
Worker crashes
             ↓
Orchestrator thinks task failed
```

```
              ↓
Retry
```

Without idempotency:

```
First execution → INSERT
Retry           → INSERT again
```

With idempotency:

```
First execution → INSERT
Retry           → recognize existing data → no duplicate
```

How do you make a pipeline idempotent?

There isn't one universal technique. Common approaches include:

1. Upsert / MERGE

Use a unique business key.

```
order_id
transaction_id
customer_id + transaction_date
```

The same record can be processed repeatedly without creating duplicates.

2. Partition overwrite

Process a deterministic partition:

```
date=2026-08-25
```

and replace that partition rather than continually appending to it.

3. Deduplication

Use an event identifier:

```
event_id = ABC123
```

If ABC123 already exists, ignore the duplicate.

4. Exactly-once-like processing

Some distributed systems provide mechanisms that help achieve exactly-once semantics, but **don't equate exactly-once processing with idempotency**. Idempotency is an application/pipeline design property; exactly-once is a broader processing/commit guarantee.

Idempotent ≠ "pipeline runs only once"

This is a very important distinction.

An idempotent pipeline **can run 10 times**.

The requirement is that:

```
Run 1 → correct result
Run 2 → same correct result
Run 3 → same correct result
```

It does **not** mean:

```
Run 1 → allowed
Run 2 → blocked
```

That's a completely different mechanism.

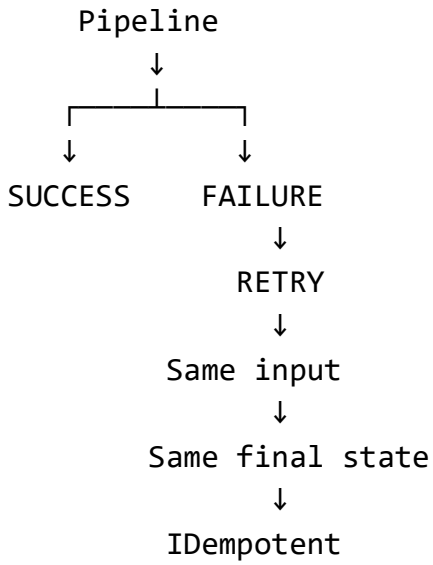
Idempotency in system design

When you're answering a DE system-design question, you can say:

"Because distributed jobs can be retried after partial failures, I'll make the load operation idempotent. I'll use a deterministic record key and MERGE/upsert semantics, or overwrite deterministic partitions for batch processing, so retries don't create duplicate records."

That demonstrates that you understand **failure handling**, not just the definition.

Mental model



The core idea is simply:

A retry should be safe.